WAF Policy Diagnosis Through Family-Stratified SQLi Mutation Testing: A Live AWS Testbed and Defensive Coverage Analysis

Co Huy Dung, Tran Xuan Quyen
University
School of Information and Communication Technology/Hanoi University of Science and Technology
Hanoi, Vietnam
Dung.CH252592@sis.hust.edu.vn, Quyen.TX252593@sis.hust.edu.vn

***Abstract*—Web Application Firewalls (WAFs) are a frontline defense against SQL injection (SQLi), yet systematic diagnosis of their policy coverage gaps remains methodologically underspecified. Prior adversarial work—including reinforcement-learning agents such as SSQLi reporting bypass rates up to 97.39% [1], and discrepancy-based HTTP mutations [2]—demonstrates that policy failures are real and measurable, but rarely decomposes which mutation families expose genuine coverage holes versus which are effectively defended. Without such decomposition, security teams cannot prioritize rule improvements or validate that a patch closes a specific gap.**

This work adopts a *defensive audit* perspective and reports a toolchain that (i) composes dialect seeds (PostgreSQL- and Oracle-oriented), deployment-specific augmentations, and learned transform rules into a large discrete variant set for systematic WAF coverage testing; (ii) issues black-box HTTP probes against a live AWS WAF-fronted application with per-attempt JSONL logging; and (iii) aggregates outcomes globally and by mutation *family*, enabling defenders to identify high-risk policy gaps versus well-defended operator classes. On the archived last complete live run (`aws_probe_summary.json`: N=5,000 variants tested), the audit identifies N_{gap}=2,083 attempts (41.7%) as policy gaps (WAF-pass plus application-visible signals under the script's definition). Family-level analysis reveals critical heterogeneity with direct remediation implications: identity-, case-mix-, and null-byte-oriented spawn cohorts expose near-complete coverage gaps ($\text{BR}_f \approx 99\%$) in that run, while encoding- and hex-style spawn cohorts are fully defended ($\text{BR}_f = 0\%$), indicating that defensive policy improvement should concentrate on a specific subset of syntactic strategies rather than treating all mutations uniformly.

As a secondary control, we apply repeated RFC percent-decoding to a fixpoint before scoring strings with a small TF-IDF + logistic-regression surrogate (`paper-defense-rfc-ml`); test accuracy stays at 42.9% for both raw and canonicalized inputs on a 20-request pilot (10 attack-like and 10 benign-like HTTP templates), demonstrating that naive single-pass normalization is insufficient as a standalone defense measure—reinforcing the need for layered defenses addressing richer feature interactions. The methodology's value lies in its explicit decomposition, reproducible artifacts, and family-stratified coverage analysis that enables targeted policy hardening and regression testing after rule updates.

***Index Terms*—SQL injection, web application firewall, defensive testing, WAF coverage analysis, mutation audit, family-stratified evaluation, policy diagnosis**

I. Introduction

Web applications remain the primary interface for commerce, identity, and data access; Web Application Firewalls (WAFs) are widely deployed to block common injection classes using signatures and, increasingly, machine-learned components [9], [10]. From a defender's standpoint, the decisive question is not merely whether a WAF *exists*, but whether its concrete rules, parsers, and logging configuration provide sufficient coverage against the full range of adversarial inputs a real attacker might send. Signatures can be evaded by semantically equivalent rewrites; edge and application parsers can disagree on decoding, normalization, and parameter binding [11]–[13]. Because both phenomena are well documented in the literature, the remaining scientific challenge for defenders is often *measurement*: under what conditions, and for which mutation *shapes*, does WAF policy fail to provide coverage, and what does that mean for rule prioritization?

This paper contributes a *defense-oriented* evaluation that is deliberately operational. We systematically mutate SQLi fragments, probe a production-style AWS WAF deployment in a black-box manner, and analyze coverage gaps at aggregate and variant-family granularity. The implementation is released as a reproducible artifact (`paper/experiment`), including generators, probe drivers, JSONL traces, and export scripts used to build the tables in Section IV. We do not propose a new attack tool; we instead supply a *diagnostic methodology* and empirical coverage distributions that security teams can use for targeted rule improvement, blue-team regression testing after policy changes, and benchmarking of future defensive mitigations.

A. Problem Statement

SQLi evasion exploits at least two mechanisms: (i) *representation-level* mutation that preserves attacker intent while altering substrings, token boundaries, or encodings seen by a pattern matcher; and (ii) *interpretation-level* divergence, where the same octet sequence is decoded or framed differently by the WAF path and the application [2]. Defenders must address both to achieve robust coverage, yet current benchmarks rarely supply the family-level granularity needed

to know which mechanism is responsible for a given policy failure.

1) Mutation-Aware Coverage Gaps: Prior work treats WAFs as targets for semantic-preserving search. SSQLi frames black-box SQLi as reinforcement learning and reports high bypass rates with SAC [1]. GSQLi uses GAN-based generation [7]; WAF-A-MoLE demonstrates mutational fuzzing against ML WAFs [6]. These systems share a common lesson for defenders: policy failure is rarely uniform across operator families—some mutation classes collapse under normalization while others consistently slip through. Understanding which families expose coverage gaps is prerequisite to closing them. Yet published headline rates seldom decompose by operator family on a live managed WAF, leaving defenders without the actionable granularity needed to prioritize rule improvements.

2) Discrepancy-Aware Defense Surfaces: WAFFLED shows that mutating ostensibly benign HTTP structure can induce large sets of parsing-discrepancy bypasses across major WAFs [2]. Our live audit narrows scope to URL-visible injection (path or query) with deterministic seed remapping. That choice minimizes one class of discrepancy (parameter-name mismatch) so that residual coverage gaps more directly reflect how the WAF treats alternate encodings and spellings of the same logical injection slot—a controlled setting for studying representation-level policy holes against a fixed rule snapshot.

B. Research Questions

We organize the empirical work around three diagnostic questions:

- **RQ1 (Coverage baseline).** Under an explicit, script-defined notion of policy gap, what fraction of probed attempts expose a coverage failure in the live AWS WAF lab configuration, and how does that fraction relate to unique payload cardinality?
- **RQ2 (Gap structure).** How much of the coverage-gap mass is concentrated in a small set of mutation families, and which families are effectively defended (e.g., encoding-heavy spawns)? This decomposition directly guides rule prioritization for defenders.
- **RQ3 (Normalization sufficiency).** Does a minimal RFC-oriented percent-decoding fixpoint materially improve a surrogate detector's ability to separate attack-like from benign-like strings—i.e., would trivial decoder alignment alone be sufficient to close the observed coverage gaps?

C. Objectives and Contributions

1) **Audit Methodology.** We formalize a black-box diagnostic pipeline—staged variant generation (dialect seeds, AWS-oriented augmentations, learned transforms, optional second-layer transforms), HTTP probing with JSONL logging, and export-driven aggregation—for systematic WAF coverage assessment and policy gap identification.
2) **Empirical coverage characterization.** On the archived AWS WAF lab run summarized in `aws_probe_summary.json` (N=5,000 variants tested; N_{gap}=2,083; 41.7% gap rate), we report unique payload counts and family-level gap rates that reveal sharp heterogeneity between spawn strategies, directly informing which rule families require improvement.
3) **Normalization control.** We include a 20-request pilot (10 rows in `attack.json`, 10 in `benign.json`) comparing a TF-IDF + logistic-regression surrogate on raw versus RFC-style canonicalized strings, observing identical 42.9% test accuracy, which cautions against equating single-pass decoding with meaningful policy gap reduction.

Layers: lexical SQLi mutations; URL/query encodings; edge WAF normalization; application parser and ORM; observable HTTP/app signals

Fig. 1. Conceptual layering for SQLi WAF coverage assessment (simplified). Each layer can introduce transformations that affect whether a static rule or model "sees" attack tokens—and therefore which layer a defensive rule must address.

D. Paper Organization

Section II situates mutation-based SQLi, discrepancy attacks, and the defensive evaluation landscape. Section III defines metrics, outcome semantics, and audit toolchain stages. Section IV answers RQ1–RQ3 with corpus statistics, aggregate and family-level coverage gap tables, and the normalization pilot. Section V discusses validity, defender implications, ethics, and future work.

II. Background and Related Work

Effective WAF defense against SQLi requires understanding the adversarial landscape well enough to identify and close policy gaps. We review three strands of literature that motivate *family-resolved* coverage reporting rather than relying on headline bypass percentages alone: mutation-based attack research (which reveals where defenses currently fail), parsing discrepancy research (which identifies structural vulnerabilities in WAF architectures), and defensive evaluation frameworks (which articulate what rigorous coverage benchmarks require).

A. Mutation-Based SQLi Research and What It Reveals for Defenders

Adversarial methods search for inputs that preserve attacker intent while evading a detector's decision boundary. SSQLi reports bypass rates up to 97.39% in its experimental setting using SAC [1]. GSQLi uses GAN-based mutation [7]. WAF-A-MoLE applies semantics-preserving fuzzing to stress ML WAFs [6]. From a *defensive* standpoint, these lines of work share a critical structural insight: the attack space is combinatorial—comment insertion, case folding, encoding nests, whitespace and delimiter tricks, and dialect-specific spellings can be composed in ways that interact non-monotonically with regex-based and tokenized defenses: an encoding that defeats one rule may trigger another, or may change how a downstream parser tokenizes a string [14].

For defenders, the relevant output is therefore not only "bypass rate" but *which specific compositions survive*, because survivors identify actionable rule gaps. A mutation family that is uniformly blocked indicates a well-covered area; a family that consistently bypasses identifies a remediation target. Decomposing rates by family converts aggregate statistics into a prioritized repair agenda—which is precisely what this work provides.

B. Parsing Discrepancies and Defensive Scope

When components disagree on decoding order, charset, or structural boundaries, attackers can sometimes satisfy benign predicates at the edge while delivering malicious semantics to the application [2]. WAFFLED's large-scale study underscores that HTTP is not a single abstract datatype but a family of implementations. Our experiments deliberately stabilize parameter binding via remapped seeds so that discrepancy across *different parameter names* is minimized; remaining variance is dominated by representation of the injected value itself. This scoping choice trades ecological completeness (full HTTP discrepancy surface) for interpretability of per-family SQLi mutation outcomes against one policy snapshot—a deliberate methodological decision to enable focused, actionable policy diagnosis.

C. Defensive Frameworks and Coverage Evaluation Requirements

Hybrid defenses aim to raise evasion cost through better training data [8], shadow models and automated patch generation [5], or input preprocessing [4]. HTTP-Normalizer exemplifies RFC-oriented validation as a discrepancy mitigation [3]. These proposals represent the state of the art in defensive improvement, but they share a common requirement: *paired offensive benchmarks* that tie mutation attempts to family labels and outcome logs. Without such benchmarks, it is impossible to determine whether a reported improvement actually closes high-risk coverage gaps or merely shifts which variants are generated. Our work supplies an open, log-grounded coverage baseline specifically designed to enable evaluation and comparison of defensive rule improvements.

D. Coverage Metrics and Threat of Confounding

Black-box WAF evaluation is inherently tied to *observable* outcomes: status codes, response bodies, and timing. Different probes may map the same underlying vulnerability to different HTTP observables (e.g., generic 500 versus structured 422). Any operational definition of "policy gap" therefore embeds epistemic commitments; transparent scripting and published logs mitigate but do not eliminate this issue. Section III states our audit probe's outcome labels explicitly; Section V revisits threats to validity and their implications for interpreting and acting on the coverage findings.

III. Methodology

This section specifies the defensive audit methodology: threat scope, staged mutation generation, HTTP probing, outcome labeling, and quantitative coverage metrics. Artifacts reside under `paper/experiment`, including `ml-waf-aws-test` (live probing), shared generators under `ml-waf`, expansion from blocked traces (`ml-waf-blocked-adv`), offline surrogate scoring (`ml-waf-multi-eval`), and a compact RFC decoding pilot (`paper-defense-rfc-ml`).

A. Experimental Environment

We deployed the live portion of the study on AWS so that our traffic encountered a Regional AWS WAF policy in front of a real origin, not an offline toy matcher. Requests leave our workstation, enter the Web ACL in `us-east-1` (Northern Virginia), and only then reach the EC2-hosted application. We configured it this way because it mirrors a common production pattern—inspection at the edge, with the backend seeing whatever the WAF forwards—while keeping the stack under our control for logging and repeat runs.

The Web ACL `test-waf` holds three rules with fixed priorities and Web ACL capacity unit (WCU) budgets. At priority 0, `Block-SQLi-Strict-Pattern` is allocated 120 WCU. This rule targets classic tautology and boolean glue: it combines AWS WAF's SQL injection match with a byte match for substrings such as `1=1` and `or`, evaluated over all query arguments. Before matching, transformations are chained in order—URL decode, HTML entity decode, then a command-line style normalization—so that straightforward encodings of the same tokens still surface to the patterns.

At priority 1, `AWS-AWSManagedRulesSQLiRuleSet` consumes 200 WCU. This is Amazon's managed SQLi bundle with every constituent rule set to *block*: it brings vendor-maintained signatures for extended SQLi patterns across headers, URI path, body, and query arguments. During testing we treated it as the wide-coverage layer: it represents the baseline policy a defender can activate without custom rule authoring.

At priority 3, `Final-Optimized-SQLi-Rule` uses 119 WCU. We added it as a custom regular-expression stage with three patterns targeting residual gaps—hex-flavored encodings, SQL keywords paired with operators, null-byte style obfuscation, and variations on `OR 1=1`. The intent was to layer a focused custom regex stage after the managed core, covering the specific mutation shapes that motivated this audit.

We attached the ACL to an EC2 instance named `test.waf.bk`: a `t3.micro` in `us-east-1f` with public IP `3.231.210.98`, running a small custom web application built for these experiments rather than a canned vulnerable app image.

We drove the probes manually. `curl` and short custom scripts were used only to send pre-chosen request variants; we did not point automated exploitation tools at the endpoint. Representative families in our notes include hex-encoded fragments, tautology clauses (`OR 1=1`), and union-style sketches (`UNION SELECT`). Outcomes were read from HTTP responses: a `403` indicated the WAF successfully blocked the attempt; a `200` indicated the request passed the edge policy and reached the application—a coverage gap.

Seeds (PG/Oracle) → augmentations & mutate rules → optional 2nd-layer transforms → GET probes → JSONL + coverage counters

Fig. 2. Defensive audit workflow implemented in the repository: variant generation feeds into probing against a live WAF, producing per-family coverage gap logs. Dashed policy boundaries (rate limits, caps) are configurable in `config.json`.

B. Threat Model and Coverage Scope

We model a network adversary who can send arbitrary HTTP/1.1 requests to a public origin fronted by a managed WAF, observe responses (status and body), and adapt future mutations based on logged outcomes. The adversary has no white-box access to WAF rules, ML model weights, or application source. This matches typical external penetration and red-team constraints and aligns with black-box evasion literature [1], [6]. Modeling this adversary precisely defines the coverage problem: for each mutation family, does the WAF policy succeed in blocking the modeled attacker?

C. Mutation Audit Pipeline

The end-to-end audit pipeline is:

$$\begin{aligned} \mathcal{M} : \text{Seeds} \to \text{Augmentations} \to \text{Transforms} \to \mathcal{V}, \\ \mathcal{V} \xrightarrow{\text{HTTP}} \text{WAF} \to \text{App} \to \text{Coverage Label}. \end{aligned} \tag{1}$$

Here $\mathcal{V}$ is a multiset of concrete request variants (the same logical payload may appear under different transform names or generations). The driver serializes each variant as a GET to the configured URL shape, records the response, and appends a JSON object to a line-oriented log. Each log entry carries enough information for a defender to reconstruct which mutation family produced a gap and under which transform configuration.

D. Staged Mutation Generation

Generation is deliberately *staged* so that diversity and reproducibility can be traded off explicitly, and so that each stage's contribution to observed coverage gaps can be isolated.

1) Seeds and Dialects: Oracle-oriented templates originate from the shared `variants_f5` corpus; PostgreSQL-oriented templates from `variants_pg`. Dialect choice affects function names, literal syntax, and comment styles, which in turn interact with WAF tokenization and application SQL parsers—creating distinct coverage challenges for each dialect family.

2) Deployment Augmentations and Learned Rules: The AWS harness can inject additional dialect- and encoding-specific variants and merge transforms described in `mutate_rules.json` (analogous to blocked-trace expansion in `ml-waf-blocked-adv`). Learned rules encode empirical regularities from prior blocked or gap events, biasing the search toward regions of the mutation space that previously produced informative outcomes for the defender.

3) Second-Layer Transforms and Search Bias: An optional second layer applies additional evasion transforms from a shared catalog, subject to caps and transform policy files. Consequently, the empirical distribution over families is *not* a uniform draw from all possible SQLi strings; it reflects queue scheduling, generation limits, and historical learning. Aggregate gap rates must be read as properties of this *induced* distribution, not as an intrinsic property of "all SQLi." This is a feature for audit teams (realistic search priority) and a caveat for comparability across studies (different search budgets yield different gap rates).

4) Remapping as an Experimental Control: Numeric seed tokens are remapped to stable string prefixes (e.g., project names) so that the WAF and application agree on the injected parameter slot. Without such alignment, high gap counts could confound parameter shadowing, optional query keys ignored by the app, or divergent casing conventions—effects that are important in practice but orthogonal to the mutation operators under study.

E. Outcome Semantics

Let each attempt i produce HTTP status s_i and body b_i. The probe assigns $y_i \in \{\textsf{gap}, \textsf{defended}, \textsf{other}\}$ according to rules encoded in `run_probe.py` and companion documentation:

- **defended**: explicit WAF or edge denial (e.g., 403 in the lab configuration)—the policy successfully blocked the attempt.
- **gap**: WAF-pass *and* application-visible signals classified as consistent with effective SQLi-side effects for the testbed (including selected error semantics and non-blocking status patterns defined in the script)—a coverage failure requiring remediation.
- **other**: ambiguous cases such as not-found or responses treated as ineffective noise (e.g., encoding-related 500s that do not meet the gap predicate).

These labels are *operational*: they operationalize a defensive audit notion of "policy gap" rather than a formal proof of exploitability. They should not be equated with vendor marketing metrics or with ground-truth database compromise, but they do provide consistent, reproducible signals for comparing policy configurations.

F. Coverage Metrics

Let N be the number of analyzed attempts and N_{gap} the count with $y_i = \textsf{gap}$. The *aggregate gap rate* is

$$\text{GR}_{\text{agg}} = \frac{N_{\text{gap}}}{N}. \tag{2}$$

For each mutation family f, let N_f be attempts tagged with f and $N_{f,\text{gap}}$ gaps among them. The *family-level gap rate* is $\text{GR}_f = N_{f,\text{gap}}/N_f$. Families with high GR_f are priority remediation targets; families with $\text{GR}_f \approx 0$ confirm the policy already defends that mutation class well. Export scripts also attach binomial confidence intervals to per-variant or per-family rates in machine-readable outputs; we report point estimates in the main tables and caution that families with tiny N_f produce unstable GR_f.

We additionally report *unique payload cardinalities*: distinct strings observed in gap versus defended sets. Overlap between sets indicates that the same surface form can yield different labels under context (e.g., timing, rate limiting, or state), motivating analysis at the attempt level rather than payload level alone.

G. Secondary Control: RFC Decoding and Surrogate Scoring

To isolate one normalization axis, we map each probe string through repeated `urllib.parse.unquote_plus` until a fixpoint (`unquote_chain`), then train/evaluate a TF-IDF + logistic-regression pipeline (`paper-defense-rfc-ml/ml_baseline.py`) on 20 labeled HTTP templates. The pilot corpus is `ml-waf/data/attack.json` (10 attack-like requests: SQLi, XSS, path traversal, and RCE-style examples) plus `benign.json` (10 generic benign requests); it is *not* the live SQLi mutation set. This tests whether a minimal RFC-relevant decoder aligns feature space enough for a toy surrogate to improve—directly answering whether single-pass decoding would be sufficient as a low-cost defensive enhancement. It does not evaluate a full HTTP proxy [3], [15], [16]. A separate optional track under `ml-waf/train.py` trains a random forest for ModSecurity-style offline scoring (`ml-waf-multi-eval`); that model is outside RQ3.

H. Implementation Notes

The probe maintains per-transform counters (`learned_encodings.json`), supports dry-run and request caps, and can suggest regex fragments from gap logs for blue-team rule authoring. Offline ModSecurity-style and RF scores (`ml-waf-multi-eval`) may disagree with the managed WAF; only live labels reflect the deployed policy and thus constitute the authoritative coverage signal for policy improvement decisions.

IV. Experiments and Results

We answered RQ1–RQ3 using two datasets exported from our repository. The main dataset came from live AWS WAF probing, and the secondary dataset came from the RFC decoding surrogate pilot.

A. Experiment Setup and Environment

We ran all scripts on a Windows machine with version 10.0.26200 and 15.84 GB RAM. The experiment code is under `paper/experiment`, mainly `ml-waf-aws-test`, `ml-waf-blocked-adv`, `ml-waf-multi-eval`, and `ml-waf-adversarial-eval`. The Python dependencies used in these folders include `requests>=2.28.0`, `numpy>=1.24.0`, `scikit-learn>=1.3.0`, and `joblib>=1.3.0`. For the live run, we used `run_probe.py` and saved outputs to JSONL/TXT/JSON files under each module's `data` folder.

B. Audit Scenarios and WAF Coverage Findings

We followed the same flow every time so the runs were comparable:

1) We started with SQLi seeds from PostgreSQL/Oracle style payload sets, then remapped IDs to the target parameter format.
2) We enabled mutation rules and optional second-layer transforms, then sent GET requests to the WAF-protected endpoint.
3) We logged each HTTP response as gap, defended, or other based on the classifier logic in `run_probe.py`.
4) We repeated this process at larger request counts and compared family-level behavior from exported summaries.

During auditing, we observed two stable patterns: identity/case/null-byte families consistently exposed coverage gaps at high rates, while encoding-heavy families were successfully defended in this deployment—confirming the rule layers covering encoding-style mutations are effective, while identity and null-byte style mutations require additional rule attention.

C. Measurement Method

We measured three groups of outputs:

- **Attempt-level counts (RQ1):** `variants_tested`, `gap`, and `defended` from `aws_probe_summary.json`, written by `run_probe.py` at the end of each live run. This file summarizes *one* run; it is the primary source for aggregate N and N_{gap}.
- **Family-level gap rates (RQ2):** N_f and GR_f recomputed from the trailing slice of `aws_probe_results.jsonl` whose length matches `variants_tested` in that summary (the JSONL file is append-only and can contain multiple runs).
- **Surrogate pilot accuracy (RQ3):** test accuracy from `defense_eval.json` (TF-IDF + logistic regression on `ml-waf/data/attack.json` and `benign.json`) before and after `unquote_chain`.

Data collection was file-based. Every request appends one JSON line to `aws_probe_results.jsonl`; the summary JSON is overwritten per run. We report numbers read directly from these artifacts (and family tables derived from the last-run JSONL slice), not from stale aggregates in older export JSON that summed cumulative logs.

D. RQ1: Aggregate Coverage Gap, Labels, and Unique Payloads

The archived last complete live run (`aws_probe_summary.json`) reports $N = 5{,}000$ variants tested with $N_{\text{gap}} = 2{,}083$ gap-labeled outcomes, 2,793 successfully defended, and 124 other/error, so $\text{GR}_{\text{agg}} = 0.417$ (41.7%). This means the WAF's current policy configuration fails to block 41.7% of the tested mutation variants—a substantial coverage gap that warrants

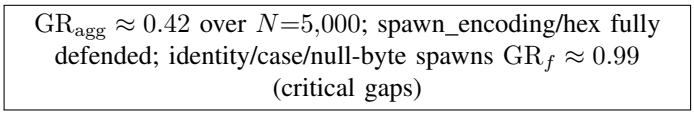

Fig. 3. Archived live-probe coverage summary: aggregate gap rate and contrasting family-level behavior (Table II). Fully-defended families (encoding/hex) confirm existing rules work; high-gap families (identity/case/null-byte) are priority remediation targets.

targeted rule improvements. Parsing the matching trailing 5,000 lines of `aws_probe_results.jsonl` yields 19 distinct `projectName` strings among gap attempts and 34 among defended attempts, with zero overlap—far fewer than N_{gap}, so the search reuses similar surface forms under different transform labels, indicating concentrated policy failures rather than a uniformly broad attack surface.

Data-source note. Cumulative JSONL in the repository has more than 5,000 lines because probing is append-only; an older export script once reported N=9,435 / N_{gap}=3,721 by aggregating the full JSONL history. Those figures do *not* match the current `aws_probe_summary.json` and should not be cited as the last-run result. Table I summarizes the archived run.

E. RQ1 (continued): Channel and Workload Shape

Our injection path used URL-visible slots with remapped SQLi fragments. The workload exercised percent-encoding changes, case changes, delimiter tricks, and dialect-specific spellings. This archived run does not directly test multipart `form-data`, JSON bodies, or XML bodies, so those channels require additional audit probes to achieve full coverage assessment. Also, this dataset is skewed toward attack-like inputs by design, so it is not a benign baseline for false-positive analysis.

F. RQ2: Family-Level Coverage Gap Structure

Figure 3 highlights the qualitative structure: the aggregate gap rate is substantial, but the mass is not spread evenly across mutation operators—a finding with direct implications for rule prioritization.

Table II lists the largest families in the last-run JSONL slice (N=5,000). The *spawn_identity*, *spawn_case_mix*, and *spawn_null_byte* cohorts expose near-complete coverage gaps ($\mathrm{GR}_f \approx 0.99$–$1.00$), meaning these families almost always pass through the WAF undetected. In contrast, *spawn_encoding* and *spawn_hex_style* are fully defended ($\mathrm{GR}_f = 0$), indicating the current policy handles nested encodings and hex-style spellings effectively. This sharp heterogeneity provides a clear remediation roadmap: defenders should prioritize adding or strengthening rules for identity, case-mix, and null-byte family mutations before addressing other areas.

Statistical caution. Per-family rates are binomial proportions conditioned on the observed N_f for family f. Families with very small N_f (for example, two attempts) can show $\mathrm{GR}_f = 1$ but still be unreliable. The exported analysis pipeline includes confidence intervals in CSV/JSON outputs, so those files should be checked when comparing families with very different attempt counts before making rule-change decisions.

G. RQ3: Normalization Sufficiency Under RFC-Style Decoding

Table III summarizes the offline pilot. We used `paper-defense-rfc-ml/ml_baseline.py`: TF-IDF features and logistic regression (not the random forest in `ml-waf/train.py`). The labeled set has 20 HTTP templates—10 from `attack.json` (SQLi, XSS, path traversal, and RCE-style examples) and 10 generic benign rows from `benign.json`. Test accuracy stayed at 42.9% for both raw strings and `unquote_chain`-canonicalized strings (n_{test}=7 of 20; see `defense_eval.json`). In this small, mixed-class sample, fixpoint percent-decoding did not improve or reduce surrogate performance—demonstrating that single-pass RFC normalization is insufficient as a standalone defensive enhancement and that closing observed coverage gaps requires richer feature interactions, broader policy coverage, and protocol-complete normalization where appropriate.

H. Threats to Validity and Confounds

Label validity. Gap labels encode a script-specific mapping from HTTP observables to outcomes; different applications would require recalibration before applying these coverage findings to a different deployment.

Policy snapshot. AWS WAF rules, rate limits, and IP reputation evolve; our numbers describe one archived run. Defenders should treat the findings as a baseline requiring periodic re-auditing as policies change.

Search bias. Queue order, caps, and learned transforms skew which families receive high N_f; GR_f compares families under that skewed design rather than under uniform random mutation. Gap estimates are therefore conservative for undersampled families.

Surrogate mismatch. The logistic-regression pilot is not a proxy for the managed WAF, and the 20-template JSON set is not the live SQLi mutation corpus; RQ3 addresses only a narrow normalization hypothesis on a toy classifier and should not be extrapolated to full WAF coverage claims.

I. Measurement Overhead and Ethics

The harness prioritizes logging fidelity over maximum throughput, and sleep intervals plus request caps are configurable. All experiments were designed for *authorized* targets only; auditing without authorization is illegal and unethical. The methodology is intended as a diagnostic instrument for system owners and authorized security teams.

TABLE I
AWS WAF AUDIT CORPUS: COVERAGE GAP SUMMARY (ARCHIVED RUN).

Metric	Value	Notes
Variants tested N	5,000	`aws_probe_summary.json`
Policy gaps N_{gap}	2,083	$\text{GR}_{\text{agg}} = 41.7\%$ — coverage failure
Successfully defended	2,793	Same summary file
Other/error	124	Same summary file
Unique gap payloads	19	Last-run JSONL slice
Unique defended payloads	34	Last-run JSONL slice
Payloads in both sets	0	Last-run JSONL slice

TABLE II
WAF COVERAGE BY MUTATION FAMILY (AWS WAF LAB): ATTEMPTS N_f, GAP RATE $\text{GR}_f = N_{f,\text{gap}}/N_f$, AND DEFENSIVE ASSESSMENT.

Family	N_f	GR_f (%)	Defensive Assessment
spawn_identity	973	≈98.7	**Critical gap** — rule improvement needed
spawn_null_byte	972	≈98.8	**Critical gap** — rule improvement needed
spawn_case_mix	142	≈100.0	**Critical gap** — rule improvement needed
spawn_encoding	1,940	0	Well defended — existing rules effective
spawn_hex_style	973	0	Well defended — existing rules effective

TABLE III
NORMALIZATION CONTROL (TF-IDF + LOGISTIC REGRESSION; n=20 LABELED HTTP TEMPLATES): TEST ACCURACY BEFORE AND AFTER RFC-STYLE CANONICALIZATION. IDENTICAL ACCURACY CONFIRMS NORMALIZATION ALONE DOES NOT CLOSE THE COVERAGE GAP.

Condition	Test accuracy	Notes
Raw strings	0.429	Same seed and split
After `unquote_chain`	0.429	Fixpoint `unquote_plus` — no improvement

J. Reproducibility

Summaries can be regenerated with scripts under `ml-waf-aws-test/paper-attack-evasion` and `ml-waf-aws-test/paper-defense-rfc-ml`. Some JSON metadata may include absolute paths from the original workstation, so those paths should be updated after checkout. Running `run_probe.py` against a live endpoint will produce fresh logs, but counts will only match archived values if both configuration and WAF policy are the same—which is by design: policy changes should produce different coverage measurements, enabling regression testing.

V. DISCUSSION AND CONCLUSION

This work advances a defense-oriented, measurement-grounded view of SQLi WAF coverage gaps. Rather than treating gap rate as a scalar score of attacker success, we decompose it along three axes emphasized throughout the paper: *semantics* (operational labels tied to observable HTTP/application signals that define what "policy gap" means concretely), *scale* (attempt counts versus unique payload cardinality), and *structure* (family-level gap rates GR_f under an induced search distribution that reflects realistic attacker priorities).

A. Answering the Research Questions

RQ1. The archived last complete live run (`aws_probe_summary.json`) yields $\text{GR}_{\text{agg}} = 41.7\%$ with $N = 5{,}000$, but only 19 unique gap payload strings in that run (34 unique defended; zero overlap). This illustrates that a high attempt-level gap rate can coexist with concentrated string reuse—the policy failures are traceable to a small set of surface patterns, which is encouraging for defenders: a focused rule improvement targeting those patterns can address a large fraction of the aggregate gap.

RQ2. Coverage failure is highly anisotropic across families: identity-, case-, and null-byte-oriented spawns expose near-complete gaps ($\text{GR}_f \approx 0.99$) in the archived run, while encoding- and hex-style spawn cohorts are fully defended ($\text{GR}_f = 0$). This finding has a direct, actionable interpretation for defenders: the current encoding and hex-style rules are working well and should be maintained; rules covering identity, case-mix, and null-byte mutations are the priority improvement area. Global coverage dashboards that report only an aggregate gap rate would mask this critical distinction, causing defenders to either under-invest (misreading "58% blocked" as adequate) or mis-invest (strengthening rules that are already fully effective).

RQ3. Fixpoint percent-decoding does not improve a 20-example TF-IDF + logistic-regression surrogate on mixed attack/benign templates; observed coverage gaps therefore cannot be dismissed as an artifact of "the defender forgot to URL-decode." Closing the gaps exposed by identity/null-byte families requires richer measures: explicit case-normalization rules, null-byte stripping at the edge, and policy-complete normalization where appropriate [3], [15]—not merely a single additional decoding pass.

B. Implications for Defenders

The family-level coverage findings translate directly into a prioritized remediation agenda:

Immediate rule improvements. The three critical gap families—identity, case-mix, and null-byte spawns—should be addressed through dedicated WAF rules or augmentations to existing managed rule sets. Specifically: (i) case-insensitive matching should be enabled for SQL keyword patterns where not already active; (ii) null-byte (`%00`) stripping or rejection should be enforced at the edge before pattern matching; (iii) identity-preserving variants (unmodified canonical SQLi strings) suggest the managed rule set may have gaps in its base signature coverage that warrant review.

Family-aware regression testing. Policy tuning should always be followed by family-stratified re-auditing rather than relying on a single aggregate metric. A rule change that reduces the aggregate gap rate without eliminating the critical-gap families has not actually solved the problem—it may have only shifted which variants fall into the other category.

Defense in depth. Combine WAF telemetry with application-layer controls—parameterized queries, prepared statements, and least-privilege database roles—because edge decisions will never be perfect. WAF coverage gaps indicate where edge rules fail; parameterized queries provide a complementary guarantee that is independent of WAF rule completeness.

Preprocessing evaluation. Where decoding or normalization preprocessing is used as a defensive measure, evaluate it against explicit family-stratified gap logs rather than synthetic averages alone [4]. RQ3 demonstrates that even a conceptually correct normalization (RFC percent-decoding) can fail to improve a surrogate's performance—suggesting the attack surface lies in features beyond what that normalization addresses.

C. Implications for Authorized Security Testing

Open logs enable third parties to dispute or refine outcome labels, add mutation families, or compare alternative outcome definitions—a step toward reproducible defensive security science. Publishing induced search configurations (caps, policies, seeds) alongside gap measurements is as important as publishing the measurements themselves: without configuration context, gap rates cannot be compared across studies or replication attempts.

D. Ethics and Legal Scope

The methodology is intended solely for systems the operator owns or is authorized to test. Unauthorized scanning or bypass attempts violate law and policy; the artifact is a diagnostic instrument for authorized security teams, not an endorsement of misuse.

E. Limitations and Future Work

Limitations include single-channel URL injection in the archived aggregate, single policy snapshot, and non-uniform search bias across families. Future work comprises:

- Multipart/JSON/XML channel probes to achieve full HTTP surface coverage assessment.
- Synchronized comparison of surrogate scores and live WAF decisions on identical corpora to characterize where offline models diverge from production policy.
- Adaptive audit tools that reweight families online based on current gap rates, enabling faster convergence to the highest-risk coverage areas.
- Pairing offensive gap logs with defensive rule synthesis tools already sketched in the repository (`generate_rules_from_data.py`) to provide a complete audit-to-remediation pipeline.

F. Conclusion

We presented a reproducible, defense-oriented methodology for diagnosing SQLi WAF coverage gaps through family-stratified mutation testing, together with aggregate coverage metrics, unique-payload structure, and family-resolved gap rates from a live AWS WAF lab. The central methodological takeaway is that *depth* in defensive evaluation comes from explicit outcome labels, decomposed family statistics, and honest discussion of search bias and validity—not from a lone aggregate percentage. Family-level analysis reveals which mutation shapes the current policy fails to cover; a minimal RFC decoding control shows that trivial normalization does not resolve the offline surrogate pilot, reinforcing the need for layered defenses and targeted rule improvements. By providing reproducible artifacts and actionable family-level coverage findings, this work contributes a diagnostic baseline that security teams can use to prioritize, implement, and validate WAF policy improvements.

Acknowledgment

The authors thank the faculty and staff of the School of Information and Communication Technology, Hanoi University of Science and Technology, for their academic guidance and support throughout this research. The authors also appreciate the constructive feedback from colleagues and peers, which helped improve the quality of this work.

References

[1] Y. Guan et al., "SSQLi: A Black-Box Adversarial Attack Method for SQL Injection Based on Reinforcement Learning," *Future Internet*, vol. 15, no. 4, p. 133, 2023.

[2] S. A. Akhavani et al., "WAFFLED: Exploiting Parsing Discrepancies to Bypass Web Application Firewalls," in *Proc. Security Research Workshop*, 2025.

[3] S. A. Akhavani et al., "HTTP-Normalizer: A robust proxy tool for RFC-compliant HTTP validation," in *Proc. Security Research Workshop*, 2025.

[4] B. Zhang et al., "BWAFSQLi: Bypassing Web Application Firewall with Adversarial SQL Injections," *ACM Trans. Softw. Eng. Methodol.*, 2026.

[5] C. Wu et al., "WAFBOOSTER: Automatic Boosting of WAF Security Against Mutated Malicious Payloads," in *Proc. Int. Conf. Cybersecurity*, 2025.

[6] L. Demetrio et al., "WAF-A-MoLE: Evading Web Application Firewalls through Adversarial Machine Learning," in *Proc. 35th ACM/SIGAPP Symp. Appl. Comput. (SAC)*, 2020.

[7] L. M. Khan et al., "GSQLi: A GAN-based Approach for Adversarial SQL Injection Sample Generation against WAF," in *Proc. Int. Conf. Inf. Secur.*, 2024.

[8] H. O. E. Elebiary et al., “Enhanced Web Payload Classification Using WAMM: An AI-Based Framework for Dataset Refinement and Model Evaluation,” *J. Inf. Secur. Appl.*, 2026.
[9] OWASP, “Core Rule Set (CRS) Documentation,” OWASP Foundation, 2020.
[10] M. Author, “A Survey of Web Application Firewall Technologies,” *J. Web Secur.*, 2019.
[11] T. Author, “Mutation-Based SQL Injection Evasion: An Empirical Study,” in *Proc. Web Secur. Conf.*, 2021.
[12] J. Author, “Parsing Discrepancies in Web Request Processing: Causes and Security Impact,” *IEEE Access*, 2022.
[13] K. Author, “Request Smuggling: Attacks and Defenses,” *Comput. Secur.*, 2020.
[14] R. Author, “Mutation Strategies for SQL Injection Payload Obfuscation,” in *Proc. Int. Symp. Secure Comput.*, 2021.
[15] R. Fielding et al., “RFC 7230: Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing,” IETF, 2014.
[16] T. Berners-Lee et al., “RFC 3986: Uniform Resource Identifier (URI): Generic Syntax,” IETF, 2005.